

才别无选择地将它设为“无任何保证”。将你的决定写成文档，这一来是为你的函数用户着想，二来是为将来的维护者着想。函数的“异常安全性保证”是其可见接口的一部分，所以你应该慎重选择，就像选择函数接口的其他任何部分一样。

四十年前，满载 `goto` 的代码被视为一种美好实践，而今我们却致力写出结构化控制流（`structured control flows`）。二十年前，全局数据（`globally accessible data`）被视为一种美好实践，而今我们却致力于数据的封装。十年前，撰写“未将异常考虑在内”的函数被视为一种美好实践，而今我们致力于写出“异常安全码”。

时间不断前进。我们与时俱进！

请记住

- 异常安全函数（`Exception-safe functions`）即使发生异常也不会泄漏资源或允许任何数据结构败坏。这样的函数区分为三种可能的保证：基本型、强烈型、不抛异常型。
- “强烈保证”往往能够以 `copy-and-swap` 实现出来，但“强烈保证”并非对所有函数都可实现或具备现实意义。
- 函数提供的“异常安全保证”通常最高只等于其所调用之各个函数的“异常安全保证”中的最弱者。

条款 30：透彻了解 `inlining` 的里里外外

Understand the ins and outs of `inlining`.

`Inline` 函数，多棒的点子！它们看起来像函数，动作像函数，比宏好得多（见条款 2），可以调用它们又不需蒙受函数调用所招致的额外开销。你还能要求更多吗？

你实际获得的比想到的还多，因为“免除函数调用成本”只是故事的一部分而已。编译器最优化机制通常被设计用来浓缩那些“不含函数调用”的代码，所以当你 `inline` 某个函数，或许编译器就因此有能力对它（函数本体）执行语境相关最优化。大部分编译器绝不会对着一个“`outlined` 函数调用”动作执行如此之最优化。

然而编写程序就像现实生活一样，没有白吃的午餐。`inline` 函数也不例外。`inline` 函数背后的整体观念是，将“对此函数的每一个调用”都以函数本体替换之。我不需要统计学博士来告诉你，这样做可能增加你的目标码（`object code`）大小。在

一台内存有限的机器上, 过度热衷 inlining 会造成程序体积太大(对可用空间而言)。即使拥有虚内存, inline 造成的代码膨胀亦会导致额外的换页行为 (paging), 降低指令高速缓存装置的击中率 (instruction cache hit rate), 以及伴随这些而来的效率损失。

换个角度说, 如果 inline 函数的本体很小, 编译器针对“函数本体”所产出的码可能比针对“函数调用”所产出的码更小。果真如此, 将函数 inlining 确实可能导致较小的目标码 (object code) 和较高的指令高速缓存装置击中率!

记住, inline 只是对编译器的一个申请, 不是强制命令。这项申请可以隐喻提出, 也可以明确提出。隐喻方式是将函数定义于 class 定义式内:

```
class Person {
public:
    ...
    int age() const { return theAge; }    //一个隐喻的 inline 申请:
    ...                                  //age 被定义于 class 定义式内。
private:
    int theAge;
};
```

这样的函数通常是成员函数, 但条款 46 说 friend 函数也可被定义于 class 内, 如果真是那样, 它们也是被隐喻声明为 inline。

明确声明 inline 函数的做法则是在其定义式前加上关键字 inline。例如标准的 max template (来自<algorithm>) 往往这样实现出来:

```
template<typename T>                                //明确申请 inline:
inline const T& std::max(const T& a, const T& b)    //std::max 之前有
{ return a < b ? b : a; }                          //关键字"inline"
```

“max 是个 template”带出了一项观察结果: 我们发现 inline 函数和 templates 两者通常都被定义于头文件内。这使得某些程序员以为 function templates 一定必须是 inline。这个结论不但无效而且可能有害, 值得深入看一看。

Inline 函数通常一定被置于头文件内, 因为大多数建置环境 (build environments) 在编译过程中进行 inlining, 而为了将一个“函数调用”替换为“被调用函数的本体”, 编译器必须知道那个函数长什么样子。某些建置环境可以在连接期完成 inlining, 少量建置环境如基于 .NET CLI (Common Language Infrastructure; 公共语言基础设施) 的托管环境 (managed environments) 竟可在运行期完成 inlining。然而这样的环境毕竟是例外, 不是通例。Inlining 在大多数 C++ 程序中是编译期行为。

Templates 通常也被置于头文件内，因为它一旦被使用，编译器为了将它具现化，需要知道它长什么样子。（这其实也不是世界一统的准则。某些建置环境可以在连接期才执行 template 具现化。只不过编译期完成具现化动作比较常见。）

Template 的具现化与 inlining 无关。如果你正在写一个 template 而你认为所有根据此 template 具现出来的函数都应该 inlined，请将此 template 声明为 inline；这就是上述 `std::max` 代码的作为。但如果你写的 template 没有理由要求它所具现的每一个函数都是 inlined，就应该避免将这个 template 声明为 inline（不论显式或隐式）。Inlining 需要成本，你不会想在没有事先考虑的情况下就招来那些成本吧。我已经提过 inlining 如何引发代码膨胀（这对 template 作者特别重要，见条款 44），但还存在其他成本，稍后再讨论。

现在让我们先结束“inline 是个申请，编译器可加以忽略”的观察。大部分编译器拒绝将太过复杂（例如带有循环或递归）的函数 inlining，而所有对 virtual 函数的调用（除非是最平淡无奇的）也都会使 inlining 落空。这不该令你惊讶，因为 virtual 意味“等待，直到运行期才确定调用哪个函数”，而 inline 意味“执行前，先将调用动作替换为被调用函数的本体”。如果编译器不知道该调用哪个函数，你就很难责备它们拒绝将函数本体 inlining。

这些叙述整合起来的意思就是：一个表面上看似 inline 的函数是否真是 inline，取决于你的建置环境，主要取决于编译器。幸运的是大多数编译器提供了一个诊断级别：如果它们无法将你要求的函数 inline 化，会给你一个警告信息（见条款 53）。

有时候虽然编译器有意愿 inlining 某个函数，还是可能为该函数生成一个函数本体。举个例子，如果程序要取某个 inline 函数的地址，编译器通常必须为此函数生成一个 outlined 函数本体。毕竟编译器哪有能力提出一个指针指向并不存在的函数呢？与此并提的是，编译器通常不对“通过函数指针而进行的调用”实施 inlining，这意味对 inline 函数的调用有可能被 inlined，也可能不被 inlined，取决于该调用的实施方式：

```
inline void f( ) {...} //假设编译器有意愿 inline “对 f 的调用”
void ( * pf ) ( )* = f; //pf 指向 f
...
f( );                  //这个调用将被 inlined，因为它是一个正常调用。
pf();                 //这个调用或许不被 inlined，因为它通过函数指针达成。
```

即使你从未使用函数指针,“未被成功 inlined”的 inline 函数还是有可能缠住你,因为程序员并非唯一要求函数指针的人。有时候编译器会生成构造函数和析构函数的 outline 副本,如此一来它们就可以获得指针指向那些函数,在 array 内部元素的构造和析构过程中使用。

实际上构造函数和析构函数往往是 inlining 的糟糕候选人——虽然漫不经心的情况下你不会这么认为。考虑以下 Derived class 构造函数:

```
class Base {
public:
    ...
private:
    std::string bm1, bm2;           //base 成员 1 和 2
};

class Derived: public Base {
public:
    Derived() { }                 //Derived 构造函数是空的,哦,是吗?
    ...
private:
    std::string dm1, dm2, dm3;     //derived 成员 1-3
};
```

这个构造函数看起来是 inlining 的绝佳候选人,因为它根本不含任何代码。但是你的眼睛可能会欺骗你。

C++ 对于“对象被创建和被销毁时发生什么事”做了各式各样的保证。当你使用 new, 动态创建的对象被其构造函数自动初始化; 当你使用 delete, 对应的析构函数会被调用。当你创建一个对象, 其每一个 base class 及每一个成员变量都会被自动构造; 当你销毁一个对象, 反向程序的析构行为亦会自动发生。如果有个异常在对象构造期间被抛出, 该对象已构造好的那一部分会被自动销毁。在这些情况中 C++ 描述了什么一定会发生, 但没有说如何发生。“事情如何发生”是编译器实现者的权责, 不过至少有一点很清楚, 那就是它们不可能凭空发生。你的程序内一定有某些代码让那些事情发生, 而那些代码——由编译器于编译期间代为产生并安插到你的程序中的代码——肯定存在于某个地方。有时候就放在你的构造函数和

析构造函数内，所以我们可以想象，编译器为稍早说的那个表面上看起来为空的 Derived 构造函数所产生的代码，相当于以下所列：

```
Derived::Derived()           // “空白 Derived 构造函数”的观念性实现
{
    Base::Base();            //初始化 “Base 成分”
    try { dm1.std::string::string(); } //试图构造 dm1。
    catch (...) {            //如果抛出异常就
        Base::~Base();       //销毁 base class 成分，并
        throw;               //传播该异常。
    }

    try { dm2.std::string::string(); } //试图构造 dm2。
    catch (...) {            //如果抛出异常就
        dm1.std::string::~string( ); //销毁 dm1,
        Base::~Base();           //销毁 base class 成分，并
        throw;                  //传播该异常。
    }

    try { dm3.std::string::string(); } //试图构造 dm3。
    catch (...) {            //如果抛出异常就
        dm2.std::string::~string( ); //销毁 dm2,
        dm1.std::string::~string( ); //销毁 dm1,
        Base::~Base();           //销毁 base class 成分，并
        throw;                  //传播该异常。
    }
}
```

这段代码并不能代表编译器真正制造出来的代码，因为真正的编译器会以更精致复杂的做法来处理异常。尽管如此，这已能准确反映 Derived 的空白构造函数必须提供的行为。不论编译器在其内所做的异常处理多么精致复杂，Derived 构造函数至少一定会陆续调用其成员变量和 base class 两者的构造函数，而那些调用（它们自身也可能被 inlined）会影响编译器是否对此空白函数 inlining。

相同理由也适用于 Base 构造函数，所以如果它被 inlined，所有替换 “Base 构造函数调用” 而插入的代码也都会被插入到 “Derived 构造函数调用” 内（因为 Derived 构造函数调用了 Base 构造函数）。如果 string 构造函数恰巧也被 inlined，Derived 构造函数将获得五份 “string 构造函数代码” 副本，每一份副本对应于 Derived 对象内的五个字符串（两个来自继承，三个来自自己的声明）之一。现在或许很清楚了，“是否将 Derived 构造函数 inline 化” 并非是个轻松的决定。类似思考也适用于 Derived 析构造函数，在那儿我们必须看到 “被 Derived 构造函数初始化的所有对象” 被一一销毁，无论以哪种方式进行。

程序库设计者必须评估“将函数声明为 inline”的冲击: inline 函数无法随着程序库的升级而升级。换句话说如果 `f` 是程序库内的一个 inline 函数, 客户将“`f` 函数本体”编进其程序中, 一旦程序库设计者决定改变 `f`, 所有用到 `f` 的客户端程序都必须重新编译。这往往是大家不愿意见到的。然而如果 `f` 是 non-inline 函数, 一旦它有任何修改, 客户端只需重新连接就好, 远比重新编译的负担少很多。如果程序库采取动态连接, 升级版函数甚至可以不知不觉地被应用程序吸纳。

对程序开发而言, 将上述所有考虑牢记在心很重要, 但若从纯粹实用观点出发, 有一个事实比其他因素更重要: 大部分调试器面对 inline 函数都束手无策。这对你应该不是太大的意外, 毕竟你如何在一个并不存在的函数内设立断点 (break point) 呢? 虽然某些建置环境勉力支持对 inlined 函数的调试, 其他许多建置环境仅仅只能“在调试版程序中禁止发生 inlining”。

这使我们在决定哪些函数该被声明为 inline 而哪些函数不该时, 掌握一个合乎逻辑的策略。一开始先不要将任何函数声明为 inline, 或至少将 inlining 施行范围局限在那些“一定成为 inline” (见条款 46) 或“十分平淡无奇” (例如 `p.135 Person::age`) 的函数身上。慎重使用 inline 便是对日后使用调试器带来帮助, 不过这么一来也等于把自己推向手工最优化之路。不要忘记 80-20 经验法则: 平均而言一个程序往往将 80% 的执行时间花费在 20% 的代码上头。这是一个重要的法则, 因为它提醒你, 作为一个软件开发者, 你的目标是找出这可以有效增进程序整体效率的 20% 代码, 然后将它 inline 或竭尽所能地将它瘦身。但除非你选对目标, 否则一切都是虚功。

请记住

- 将大多数 inlining 限制在小型、被频繁调用的函数身上。这可使日后的调试过程和二进制升级 (binary upgradability) 更容易, 也可使潜在的代码膨胀问题最小化, 使程序的速度提升机会最大化。
- 不要只因为 function templates 出现在头文件, 就将它们声明为 inline。